

Station-Level Bike Rental Demand Forecasting with Lag Features and Ensemble Regression

Anton Wilhelmi and Alexander Harant

Introduction to Machine Learning

Group Project – Final Deliverable

TABLE I
PROBLEM FORMULATION AND DESIGN CONSTRAINTS.

Element	Final formulation
Learning type	Supervised learning, regression.
Prediction target	Daily rentals per station: <code>total_rentals</code> .
Observation unit	One row equals one start station on one calendar day.
Practical goal	Support operational planning by estimating future station-level demand.
Main constraint	Time order must be respected; future data cannot influence training.
Main risk	Leakage through rental sub-counts or lag features computed across stations.
Model choice principle	Start with simple baselines, then add complexity only when results justify it.

Abstract—This project predicts daily bike rental demand for the 20 most used Capital Bikeshare start stations. The problem is formulated as supervised regression with `total_rentals` as target. The main contribution is not only the final model score, but the structured path from raw data to a reproducible modelling pipeline: raw trip, station, and weather data are aggregated to station-day level; leakage-prone count decompositions are removed; redundant weather predictors are reduced using correlation analysis; station identity is represented with one-hot encoding; and exact station-specific lag features are added. All models are evaluated with the same chronological train-validation-test split and the same metrics. The final best model is Gradient Boosting with lag features, reaching a test RMSE of 22.78, MAE of 16.92, and R^2 of 0.663. The clearest result is that temporal representation matters most: adding lag features improves test RMSE for every evaluated model.

I. INTRODUCTION

Bike-sharing systems depend on local and time-dependent demand. If too few bikes are available, users cannot start trips; if too many bikes accumulate at one station, docks and redistribution capacity are wasted. We therefore study the following machine learning question: given station, calendar, weather, and recent demand information, how accurately can daily bike rentals be predicted for a station?

The task is a supervised regression problem because the output is a numeric rental count. The final target is `total_rentals`. The main input groups are station identity, weather, calendar variables, chronological order, and lagged demand. The project deliberately focuses on a transparent pipeline rather than a single black-box result. Each data transformation is justified because the final model quality depends strongly on representation choices.

The final report follows the project requirements: related work, data description, preprocessing, modelling strategy, eval-

uation, results, limitations, critical reflection, and AI tool acknowledgement. The paper also reflects the feedback from earlier deliverables: stronger dataset description, explicit metrics, careful handling of imbalance across demand levels, and more critical analysis of feature importance and overfitting.

II. RELATED WORK AND COMPARABLE PROJECTS

Bike rental demand forecasting is a common regression benchmark because rental counts are influenced by calendar patterns, weather, and local mobility habits. The older UCI Bike Sharing Dataset uses Capital Bikeshare data from 2011–2012 and is widely used for hourly or daily rental prediction [3]. Kaggle’s Bike Sharing Demand competition uses a similar forecasting framing and shows that engineered time and weather features are standard for this task [4]. Our project differs by using a newer Capital Bikeshare dataset from 2020/05–2024/08 and by modelling demand at station-day level instead of city-wide hourly demand [1], [2].

The reviewed work shows that strong demand forecasts usually depend on both model choice and feature construction. This is why the paper gives more space to the pipeline decisions than to a long list of final plots.

III. DATASET CONSTRUCTION AND USED FEATURES

The raw project data contains trip usage information, station metadata, and weather data. The final learning table is not the raw trip table. It is a reduced and reproducible station-day dataset: one row means one start station on one calendar day, and the target is the total number of rentals started at that station on that day. This reduction is necessary because it aligns trip counts with daily weather information, gives a clear target variable, and avoids mixing row granularities.

A. Target Variable and Distribution

The target was simplified to one output, `total_rentals`. Intermediate count columns for member/casual users or bike types were removed from the predictor set because they are components of the same total. Keeping them would create a leakage-like shortcut: the model could reconstruct demand instead of forecasting it.

B. Why the Top 20 Stations Were Used

The top-20 filter is a methodological reduction. Rare stations contain fewer repeated observations, which makes station-specific patterns unstable and weakens evaluation. Highly used

TABLE II
COMPARABLE WORK AND WHAT WAS ADOPTED OR AVOIDED IN THIS PROJECT.

Source	Problem and data	Methods / focus	Consequence for our design
UCI Bike Sharing Dataset [3]	Hourly and daily Capital Bikeshare demand from 2011–2012.	Regression benchmark using weather and calendar variables.	Confirms the task type and feature groups, but the data is older and less station-specific than our final setup.
Kaggle Bike Sharing Demand [4]	Forecast rental demand using historical usage and weather information.	Many solutions rely on time features, weather variables, and nonlinear regression.	Supports a regression pipeline with feature engineering instead of a purely raw-table approach.
Capital Bikeshare open data [2]	Public trip and system data for the Washington, D.C. bike-sharing system.	Operational data source for station-level demand construction.	Justifies our station-day aggregation and the focus on real operational station behaviour.
owenpb GitHub project [5]	Bike-sharing demand prediction.	Linear, Lasso, Ridge, KNN, Decision Tree, Random Forest, AdaBoost, XGBoost.	Motivates comparing multiple model families rather than reporting only one algorithm.
nicolasguillaume GitHub project [6]	Kaggle bike-sharing demand forecasting.	Weather/time features and Random Forest.	Supports tree ensembles as a reasonable candidate for nonlinear demand patterns.
gradinarualex GitHub project [7]	UCI bike rental prediction.	Baseline first, then feature engineering and stronger models.	Matches our structure: baseline, data representation, then model comparison.

TABLE III
FINAL DATASET DEFINITION.

Item	Value / decision
Source	Capital Bikeshare dataset 2020/05–2024/08 [1].
Raw inputs	Trip records, station list, weather table.
Selected stations	Top 20 start stations by total rentals.
Observation level	Daily start-station demand.
Target variable	total_rentals.
Initial reduced rows	31,289 station-day rows.
Comparable rows	30,840 rows after filtering lag and no-lag variants to identical station-day keys.
Train / validation / test	21,410 / 4,705 / 4,725 rows.
Encoded features	35 without lag; 37 with lag_1 and lag_7.
Split logic	Chronological 70/15/15 split.

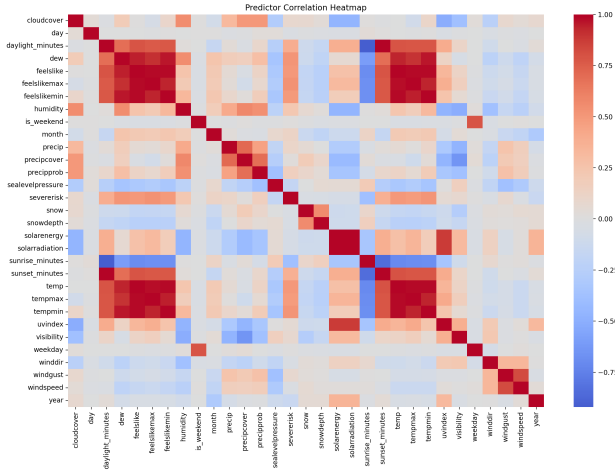


Fig. 1. Correlation matrix used during feature reduction. Strong blocks identified near-duplicate weather variables and supported removing redundant predictors before modelling.

stations provide enough examples for model comparison and reduce random noise. The limitation is explicit: the final model represents busy stations, not the complete bikeshare system.

IV. FEATURE ENGINEERING AND DATA DECISIONS

The feature engineering follows the order in the handwritten project plan: reduce the data, define the target, select features,

encode categorical information, decide about scaling and PCA, then add lag features after the first modelling results showed missing temporal memory.

A. Station Identity and One-Hot Encoding

The first implementation treated `start_station_id` as a numeric column. This was practical but conceptually wrong: station 31623 is not larger than station 31245 in a meaningful modelling sense. The final version therefore uses one-hot encoding. This is especially important for Linear, Ridge, Lasso, KNN, and Neural Network models because numeric station IDs would create artificial distances or coefficients. Tree models can sometimes split numeric IDs, but one-hot encoding gives a cleaner and comparable representation across all model families.

B. Scaling and PCA

Scaling is not a universal preprocessing step. It is used only where the model logic needs it. KNN depends on distances, regularized linear models depend on coefficient penalties, and Neural Networks depend on gradient-based optimization. Decision Tree, Random Forest, and Gradient Boosting do not need scaling because their splits are threshold-based. PCA was rejected because the reduced feature set is already manageable and the paper needs interpretable decisions. A principal component may be statistically useful, but it would no longer clearly mean temperature, weekday, station, or lagged demand.

C. Lag Features

Lag features are computed station-wise, not globally. A global shift would mix demand from different stations and create wrong information. The correct lag for a station-day is the same station’s demand one day before or seven days before. Rows without exact lag history are removed. The same rows are also removed from the no-lag variant so that comparisons remain fair. This step directly addresses the main weakness of the preliminary no-lag models: they used current weather and calendar information, but no memory of recent station demand.

TABLE IV
FEATURES USED IN THE FINAL MODELLING DATASET. COUNT DECOMPOSITIONS ARE NOT USED AS INPUTS.

Feature group	Variables	Why used
Station identity	start_station_id $\rightarrow$ 20 one-hot station columns	Captures persistent station-specific demand differences without treating IDs as ordered numbers.
Weather level	tempmax, humidity, sealevelpressure, visibility, uvindex	Describes general daily riding conditions. Temperature and visibility usually affect comfort and willingness to ride.
Bad-weather / surface conditions	precip, precipcover, cloudcover, windspeed, snow, snowdepth	Captures conditions that can reduce bike usage or create unusual demand days.
Calendar and time	month, weekday, year, time_idx, sunset_minutes	Preserves seasonality, weekday patterns, long-term order, and daylight information.
Temporal memory	total_rentals_lag_1, total_rentals_lag_7	Adds station-specific demand history: yesterday and the same weekday one week earlier.
Target only	total_rentals	Output variable; never used as current-day input, only its past values are allowed as lag features.

TABLE V
TARGET DISTRIBUTION AFTER FINAL COMPARABLE FILTERING.

Statistic	total_rentals
Mean	73.77 rentals per station-day.
Median	70 rentals.
Standard deviation	39.75 rentals.
Minimum / maximum	1 / 377 rentals.
Interpretation	Demand contains low, normal, and high-demand days. Therefore MAE alone is insufficient; RMSE and relative errors are also needed.

D. Why Representation Was Prioritized Over More Algorithms

Adding more algorithms would not solve a weak data representation. The early results showed that nonlinear models were better than linear models, but the largest conceptual weakness was missing temporal memory. Therefore, the final development focused on station encoding and lag features before interpreting the final ranking. This follows a data-centric approach: improve the input representation first, then compare models under one consistent protocol.

V. MODELLING PIPELINE

All models use one shared pipeline structure. This is important because a model comparison is only meaningful when models receive the same rows, target, split, preprocessing rules, and metrics. The repository therefore separates common utilities from model-specific scripts.

A. Chronological Split

A random split would mix past and future observations. This would make the task easier than real forecasting and could hide temporal drift. The final split is therefore chronological: 70% training, 15% validation, and 15% test. The validation set is used for development decisions and hyperparameter tuning. The test set is used only for final reporting.

VI. MODEL SELECTION STRATEGY

The model set was chosen to cover increasing complexity and different learning principles taught or discussed in the course: baseline modelling, linear regression, regularization,

local neighbourhood methods, decision trees, bagging, boosting, and a nonlinear neural reference. This is more informative than testing many similar variants of one algorithm.

A. Overfitting Logic

The model comparison also serves as an overfitting diagnosis. Linear models are expected to be stable but may underfit. KNN and single trees can memorize local patterns and therefore risk high variance. Random Forest combats variance through bagging. Gradient Boosting can reduce bias, but must be tuned carefully because too many or too deep trees can overfit. This is why every model is evaluated on train, validation, and test data and why the final choice is not based on training error.

VII. EVALUATION STRATEGY

The main ranking metric is RMSE because large demand misses are operationally more problematic than small errors. However, one metric is not enough. The final comparison reports MAE, RMSE, MAPE, R^2 , explained variance, and training/prediction time.

A. Overfitting Diagnosis

Overfitting is assessed by comparing train, validation, and test behaviour. KNN and single trees are expected to be more vulnerable because they can memorize local or threshold-specific patterns. Regularized linear models are safer but may underfit nonlinear demand structure. Ensemble trees are expected to balance flexibility and generalization, but they still require validation because boosting can overfit when depth and number of estimators are too large.

B. Hyperparameter Tuning

Hyperparameters were tuned on the validation set only. The test set was kept untouched until the final comparison. This separation is necessary because selecting parameters directly on the test set would turn the test set into another development set and make the final score too optimistic.

TABLE VI
MAIN PREPROCESSING AND REPRESENTATION DECISIONS.

Decision	Implementation	Justification
Station-day aggregation	Group trips by <code>start_station_id</code> and date, then merge daily weather.	Gives one consistent observation unit and matches the daily weather granularity.
Feature choice	Keep station identity, selected weather variables, calendar/time variables, and later lag features.	These groups directly match the demand mechanism: location, conditions, seasonality, and recent usage.
Human labels separated	Store station names in a mapping file instead of using names as predictors.	Names help interpretation but duplicate station identity and are not numeric model inputs.
Date transformation	Replace raw date strings with <code>month</code> , <code>weekday</code> , <code>year</code> , and <code>time_idx</code> .	Preserves calendar and order information in numeric form.
Leakage control	Remove member/casual and bike-type count decompositions from input features.	These variables are components of the target and would make prediction unrealistically easy.
Correlation reduction	Use Fig. 1 to remove near-duplicates such as <code>solarenergy/solarradiation</code> and <code>feelslike/temp</code> .	Reduces redundant predictors and improves interpretability, especially for linear models.
No PCA	Keep original reduced predictors instead of principal components.	The feature set is already small; PCA would hide the physical meaning of weather, time, and station effects.
One-hot station encoding	Encode <code>start_station_id</code> as binary station columns.	Station IDs are nominal categories, not ordered quantities. One-hot encoding creates one binary feature per category and avoids false numeric distances [8].
Selective scaling	Standardize only Linear, Ridge, Lasso, KNN, and Neural Network inputs.	These models use coefficients, penalties, distances, or gradient optimization. Tree models split by thresholds and do not require scaling.
Exact lag features	Add <code>total_rentals_lag_1</code> and <code>total_rentals_lag_7</code> per station.	The first no-lag results showed that weather and calendar alone miss temporal dependence.
Fair lag/no-lag comparison	Filter both variants to the same station-day keys.	Prevents a false improvement caused by different rows rather than different features.

TABLE VII
PIPELINE STAGES AND PURPOSE.

Stage	Purpose
Raw data loading	Read trip, station, and weather files.
Station-day table	Aggregate rentals and merge weather by date.
Feature reduction	Remove leakage columns and redundant predictors.
Lag/no-lag variants	Create two comparable datasets with identical row keys.
Encoding	Apply one-hot encoding to station identity.
Model-specific scaling	Scale only models that require it.
Chronological split	Use older data for training and later data for validation/test.
Training and tuning	Fit all models first, then optimize hyperparameters on validation data.
Evaluation	Save metrics, predictions, rankings, and the final comparison figure.

VIII. RESULTS

The final comparison contains two variants for each model: without lag and with lag. This design directly answers the central question from the development process: is the improvement mainly caused by a more complex algorithm or by better temporal information?

Hyperparameter tuning only slightly improved the strongest models. This is an important result: the main performance gain came from better representation, especially lag features, not from forcing a complex parameter search.

A. Result Interpretation

Gradient Boosting with lag features is the best final model. It reaches test RMSE 22.78, MAE 16.92, and R^2 0.663. Random Forest with lag is close, and Lasso with lag is the strongest linear model. This ranking gives a useful interpretation: nonlinear ensembles are best when maximum accuracy is

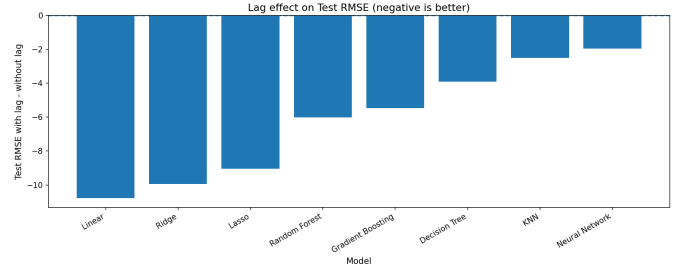


Fig. 2. Most informative final result: test RMSE decreases for every model when station-specific lag features are added. This directly supports the decision to prioritize temporal representation over adding more unrelated algorithms.

required, but a simple regularized model becomes competitive once the dataset contains the right temporal features.

The most important result is not only the winning model. Lag features improve every model. The improvement is largest for linear models because lag variables add information that calendar and weather features could not express. Ensemble trees remain strongest because they can combine station, weather, calendar, and lag information through nonlinear splits.

IX. DISCUSSION AND CRITICAL REFLECTION

The strongest lesson from the project is that representation decisions mattered more than simply adding more algorithms. Before lag features, even flexible models still missed a large part of the demand structure. After adding exact station-specific lags, every model improved. This means the project is

TABLE VIII
MODELS, USED INPUT VARIANT, SCALING, AND REASON FOR INCLUSION.

Model	Data used	Scaling	Reason for inclusion
Dummy Regressor	no-lag baseline	no	Minimum reference; real models must clearly beat it.
Linear Regression	no-lag and lag	yes	Tests whether the selected features contain an approximately linear signal.
Ridge	no-lag and lag	yes	L2 regularization checks whether coefficient shrinkage helps under remaining collinearity.
Lasso	no-lag and lag	yes	L1 regularization adds feature-selection behaviour and gives a transparent benchmark.
Decision Tree	no-lag and lag	no	Tests nonlinear threshold rules; useful but prone to overfitting.
KNN	no-lag and lag	yes	Tests whether similar station-days have similar demand; distance model needs scaling.
Random Forest	no-lag and lag	no	Bagging ensemble reduces single-tree variance and gives a strong nonlinear baseline.
Gradient Boosting	no-lag and lag	no	Sequential trees fit residual structure and target remaining bias.
Neural Network	no-lag and lag	yes	Flexible nonlinear reference; included to compare against tree ensembles.

TABLE IX
EVALUATION METRICS AND REASON FOR USE.

Metric	Interpretation in this project
MAE	Average absolute error in rentals; easy operational interpretation.
RMSE	Penalizes large errors; main ranking metric.
Median AE	Typical robust error; less affected by rare extreme days.
MAPE	Relative error; useful because stations differ in demand level.
R^2	Variance explained compared with a mean baseline.
Explained variance	Checks whether predicted spread follows real demand spread.
Fit / prediction time	Shows whether accuracy gains have a computational cost.

not mainly a story about one winning algorithm; it is a story about converting a raw mobility dataset into a representation that matches the forecasting problem.

A. Why Gradient Boosting Wins

Gradient Boosting performs best because it combines weak regression trees sequentially and focuses later trees on remaining errors. This is useful for bike demand because effects are not purely additive. Rain, weekday, station identity, and previous demand can interact: a rainy weekday at a commuter station may behave differently from a rainy weekend at a leisure station. A linear model can only capture such interactions if they are explicitly engineered. Gradient Boosting can approximate many of them through tree splits.

B. Why the Result Is Still Not Perfect

A test R^2 of 0.663 is useful but not complete. Around one third of the variance remains unexplained. Some reasons are visible from the data design. The model does not include public events, strikes, road works, station outages, local tourism, or detailed hourly weather changes. Daily aggregation also hides commuting peaks and short-term demand shocks. Therefore, remaining errors are not only model errors; they also reflect missing explanatory information.

C. Overfitting and Underfitting

The model family comparison gives a clear diagnosis. Linear models without lag underfit because they cannot describe the temporal structure. KNN and single trees can fit local structure, but they are more sensitive to variance and specific samples. Ensemble trees reduce this problem: Random Forest reduces variance by averaging trees, while Gradient Boosting reduces bias by fitting residuals. The final results therefore fit the expected bias-variance behaviour from the course.

D. Ethical and Responsible Use

The task is low-risk compared with personal classification tasks because the data is aggregated and no individual user is predicted. Still, operational bias is possible. If the model is trained only on popular stations, it may support decisions that favour already busy areas. In a real deployment, forecasts should be monitored by neighbourhood and station type. The model should support planners, not replace human judgement.

E. Next Steps

The most useful next steps are data improvements rather than simply adding more models. First, add hourly modelling to capture commute peaks. Second, add holiday and event information. Third, analyse residuals by station, season, and demand level. Fourth, monitor drift because mobility patterns can change over time. Fifth, extend the station set carefully so that less frequent stations are represented without destroying model stability.

X. CONCLUSION

This project built a complete station-level bike rental demand forecasting pipeline. The final model predicts daily rentals for the 20 most used Capital Bikeshare start stations. The most important technical decisions were: remove leakage-prone count decompositions, reduce redundant weather variables using correlation analysis, one-hot encode station identity, respect chronological order, and add exact station-specific lag features.

The best final model is Gradient Boosting with lag features, with test RMSE 22.78, MAE 16.92, and R^2 0.663. The most important insight is not only that Gradient Boosting wins, but why it wins: the final representation gives the model station identity, weather context, calendar structure, and recent demand memory. Lag features improved every model, which shows that temporal information is essential for this forecasting task.

Future work should move from daily to hourly forecasting, add event and holiday context, and evaluate fairness across all stations rather than only the most used ones. This would make the model more realistic for operational planning.

AI TOOL USE

AI tools were used for drafting support, language refinement, source search, and LaTeX structuring. They were

TABLE X
FINAL TEST RESULTS AFTER HYPERPARAMETER TUNING. LOWER RMSE/MAE/MAPE IS BETTER; HIGHER R^2 IS BETTER.

Model	Variant	Features	MAE	RMSE	R^2	MAPE	Fit s	Pred. s
Dummy	baseline	–	38.14	48.02	-0.499	0.817	–	–
Linear	no lag	35	27.67	35.75	0.169	0.613	0.017	0.001
Linear	lag	37	19.04	24.97	0.595	0.340	0.010	0.001
Ridge	lag	37	19.00	24.92	0.596	0.345	0.002	0.001
Lasso	lag	37	17.78	23.86	0.630	0.291	0.013	0.001
Decision Tree	lag	37	19.47	26.68	0.537	0.282	0.074	0.002
KNN	lag	37	19.55	25.37	0.582	0.282	0.001	2.696
Random Forest	lag	37	17.17	23.02	0.656	0.295	0.322	0.060
Gradient Boosting	lag	37	16.92	22.78	0.663	0.253	9.567	0.060
Neural Network	lag	37	18.89	24.31	0.616	0.299	6.021	0.017

TABLE XIII
EFFECT OF LAG FEATURES ON TEST RMSE.

Model	No lag	Lag	Improvement
Linear	35.75	24.97	30.2%
Ridge	34.87	24.92	28.5%
Lasso	32.91	23.86	27.5%
Random Forest	29.02	23.02	20.7%
Gradient Boosting	28.24	22.78	19.3%
Decision Tree	30.58	26.68	12.8%
KNN	27.86	25.37	9.0%
Neural Network	26.26	24.31	7.4%

TABLE XIV
MAIN LIMITATIONS AND PRACTICAL CONSEQUENCE.

Limitation	Consequence
Top-20 station focus	Excludes smaller stations; results should not be generalized to the complete system.
Daily aggregation	Removes hourly rush-hour peaks and short weather changes.
Missing event variables	Large peaks or drops caused by events may remain unexplained.
Chronological drift	Later demand behaviour can differ from earlier training data.
High-demand days	The model may still underestimate rare high-demand situations.
Weather-only context	Social, infrastructure, and operational factors are mostly absent.

TABLE XI
FINAL TOP THREE MODELS AND WHY THEY ARE DISCUSSED.

Model	Reason
Gradient Boosting + lag	Best final accuracy: RMSE 22.78 and R^2 0.663.
Random Forest + lag	Nearly equal accuracy, faster training, robust bagging alternative.
Lasso + lag	Best linear model; useful transparent benchmark against ensembles.

TABLE XII
HYPERPARAMETER TUNING EFFECT FOR THE FINAL TOP THREE MODELS.

Model	RMSE before	RMSE after	R^2 before	R^2 after
Gradient Boosting + lag	22.79	22.78	0.662	0.663
Random Forest + lag	23.02	23.02	0.656	0.656
Lasso + lag	23.89	23.86	0.629	0.630

also used to help check whether the final report structure follows the project guidelines. The dataset construction, code execution, model outputs, final methodological choices, and interpretation were reviewed and controlled by the authors.

REFERENCES

- [1] T. Lo, “Capital bikeshare dataset 2020/05–2024/08,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/taweilo/capital-bikeshare-dataset-202005202408>
- [2] Capital Bikeshare, “System Data.” [Online]. Available: <https://capitalbikeshare.com/system-data>
- [3] H. Fanaee-T, “Bike Sharing,” UCI Machine Learning Repository, 2013. [Online]. Available: <https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset>
- [4] Kaggle, “Bike Sharing Demand.” [Online]. Available: <https://www.kaggle.com/competitions/bike-sharing-demand>
- [5] O. B., “Kaggle-Bike-Sharing-Prediction,” GitHub. [Online]. Available: <https://github.com/owenpb/Kaggle-Bike-Sharing-Prediction>
- [6] N. F. Guillaume, “Bike-Sharing-Demand-Kaggle,” GitHub. [Online]. Available: <https://github.com/nicolasfguillaume/Bike-Sharing-Demand-Kaggle>
- [7] A. Gradinaru, “bike-rental-predictions,” GitHub. [Online]. Available: <https://github.com/gradinarualex/bike-rental-predictions>
- [8] scikit-learn, “OneHotEncoder.” [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>